Nadel · payment accounting for garden cooperatives

Platform owner's manual

How the platform is built and how to run it: components, running it locally, Keycloak, creating cooperatives and users, the platform operator, every screen and button of the cooperative admin, data security, server deployment, development, common problems.

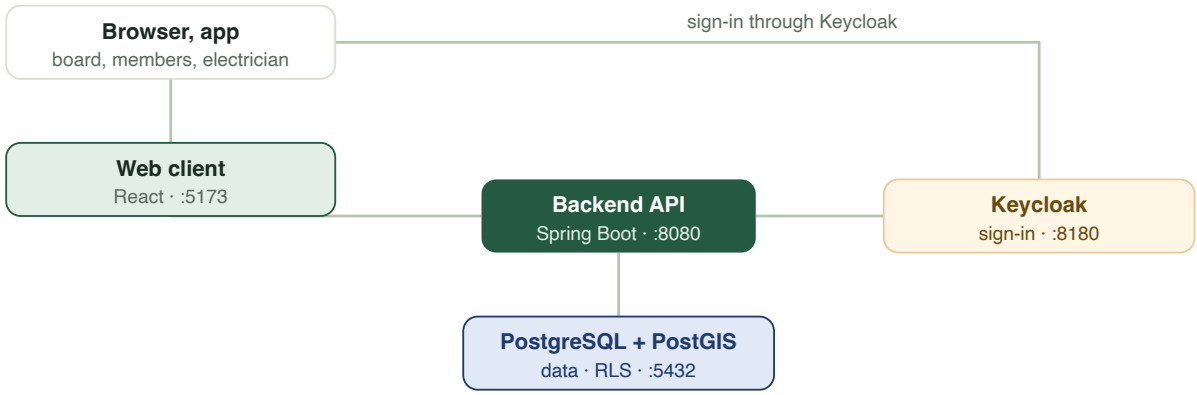
Reader	The platform owner and administrator
State	Chapters marked "works" describe the system as it is; "planned" — decided but not built yet
Edition	22 October 2026

Contents

1	How the platform is built	works
2	Local environment	works
3	Keycloak: sign-in and accounts	works
4	Cooperatives: creation and isolation	works
5	The platform operator	partly
6	Cooperative admin: screens and buttons	works
7	Permissions and roles	works
8	Data, security, personal data	works
9	Server deployment	planned
10	Updates and database migrations	works
11	How the system is developed	works
12	Common problems and fixes	works
13	Decision log	

1 How the platform is built

One platform serves many cooperatives. Each cooperative's data is kept apart by the database itself.



Platform components and local ports

Web client	The board's screens and the member's account. It calculates nothing; it shows what the server returns.
Backend	One application (a modular monolith) with the modules `tenancy` (cooperatives), `identity` (sign-in and permissions), `members`, `plots`, `settings`, `imports`, `tariffs`, `meters`, `money`, `audit`. A test guards the module boundaries.
API contract	`api/openapi.yaml` is the single source of truth: the server interfaces and the web and app client are generated from it.
PostgreSQL	All data. Every table with cooperative data has a `tenant_id` and an RLS policy: a query only sees its own cooperative's rows.
Keycloak	Accounts, passwords, sign-in. Cooperative roles are not kept in Keycloak but in the database; Keycloak holds only the platform operator role.
Mobile app	An Expo shell exists; the real app is phase 5.

How a request flows

1. A person signs in through Keycloak and gets a token with the `tenant_id` attribute (their cooperative).
2. The backend checks the token, finds the member by email and takes their permissions from the database.
3. For the request the database receives the `tenant_id`; RLS lets through only that cooperative's rows.

4. Board changes are published as events; the audit module records them through an "outbox", so no event is lost even if the server restarts.

2 Local environment

What the computer needs

Docker Desktop, JDK 25, Node.js 22 and pnpm, Git, the GitHub CLI (`gh`). Repository: `~/IdeaProjects/nadel-platform`.

First run

```
make install        # web and shared package dependencies
make up             # database, Keycloak, backend and web in one terminal (Ctrl+C stops
backend and web)
make e2e-install    # once: the browser for end-to-end tests
```

Make commands

Command	What it does
`make help`	Lists every command
`make install`	Installs client dependencies (pnpm)
`make infra`	Starts PostgreSQL + PostGIS and Keycloak in Docker and waits until ready
`make up` / `make dev`	Infrastructure + backend (:8080) + web (:5173)
`make backend`, `make web`, `make mobile`	The backend, the web or the Expo server alone
`make down`	Stops the Docker containers (data is kept)
`make gen`	Regenerates the API client from `api/openapi.yaml`
`make test`	Backend tests (with a database container) and client tests
`make check`	Everything CI checks: contract, formatting, lint, tests
`make e2e`	End-to-end browser scenarios (Playwright) on a separate database
`make guide-shots`, `make guide`	Screenshots for the guides (from the running app) and the PDF build
`make clean`	Removes build output

Addresses and passwords (local development only)

What	Address	Sign-in
The app	`http://localhost:5173`	demo users below
API	`http://localhost:8080/api/v1`	a Keycloak token
Keycloak admin console	`http://localhost:8180/admin/`	realm master, `admin` / `admin`
PostgreSQL	`localhost:5432`, database `nadel`	`postgres` / `postgres` (superuser), `nadel_owner`, `nadel_app`

Demo users (password `nadel`)

Email	Who
`chairman@demo.local`	Chairman of CT «Надія»
`treasurer@demo.local`	Treasurer
`electrician@demo.local`	A member with the custom role "Електрик"
`member@demo.local`	A member
`member@other.local`	A member of another cooperative (CT «Світанок»), to check isolation
`operator@nadel.local`	The platform operator (no access to cooperative data)

Resetting the local database

```
make down && docker compose -f infra/docker-compose.yml down -v && make up
```

Erases all local data; migrations and demo data load again and Keycloak imports the realm again. Needed after changing `infra/keycloak/nadel-realm.json` or when test data has spoiled the demo.

3 Keycloak: sign-in and accounts

Keycloak keeps accounts and passwords. The platform's realm is `nadel` with two clients, `nadel-web` and `nadel-mobile`. Sign-in is by email, self sign-up is off, password reset by email is allowed (it works once mail is configured), the sign-in page speaks Ukrainian (default) and English.

How an account is tied to a cooperative

- The user attribute `tenant_id` is the cooperative's identifier. It goes into the token, and the backend works only with that cooperative's data.
- The member is found by **email**: the email in Keycloak must match the email on the member card. The backend remembers the link at the first sign-in.
- Roles and permissions live in the cooperative's database, not in Keycloak. The exception is the `PLATFORM_OPERATOR` role.

Creating an account for a member

1. Make sure the member exists in the system and their card has an email (the chairman does this).
2. Keycloak admin console → choose the realm **nadel** at the top left → **Users** → **Add user**.
3. Username and Email — the same address; First name, Last name; switch on **Email verified**; **Create**.
4. Tab **Attributes**: key `tenant_id`, value — the cooperative's identifier (chapter 4) → **Save**.
5. Tab **Credentials** → **Set password**: a temporary password with **Temporary** on — the person sets their own at the first sign-in.

> **NO ATTRIBUTES TAB?**
>
> Keycloak 26 hides custom attributes by default. Once: **Realm settings** → **General** → **Unmanaged attributes** → "Admin can edit" → **Save**.

Other tasks

Task	Where
Reset a password	Users → user → Credentials → Reset password (Temporary)
Block sign-in	Users → user → switch off **Enabled**. Archiving the member in the system also blocks sign-in.
End all sessions	Users → user → Sessions → Sign out

Task	Where
Make someone the platform operator	Users → user → Role mapping → Assign role → `PLATFORM_OPERATOR`; no `tenant_id` attribute
Change realm settings for good	Edit `infra/keycloak/nadel-realm.json` and reset the local database (chapter 2): the file is imported only into an empty Keycloak

Creating accounts and email invitations from the cooperative admin come with the operator console and mail (phases 4 and 7). Until then the platform owner creates accounts in Keycloak.

4 Cooperatives: creation and isolation

A cooperative (a *tenant* in the code) is a row in the `tenant` table with an identifier (UUID) and a name. When a cooperative is created the system gives it the base roles, the settings and the standard charge types.

Creating a new cooperative (until the operator console exists)

1. Connect to the database as the superuser: `docker compose -f infra/docker-compose.yml exec postgres psql -U postgres -d nadel`.
2. Run, with the name and the chairman's details filled in:

```
DO $$
DECLARE t uuid := gen_random_uuid(); m uuid := gen_random_uuid();
BEGIN
  INSERT INTO tenant (id, name) VALUES (t, 'СТ «Назва»');
  PERFORM seed_base_roles(t);          -- roles, settings, charge types
  INSERT INTO member (id, tenant_id, last_name, first_name, email, delivery_channel)
  VALUES (m, t, 'Last name', 'First name', 'chairman@example.com', 'EMAIL');
  INSERT INTO member_role (tenant_id, member_id, role_id)
  SELECT t, m, id FROM role WHERE tenant_id = t AND base_code = 'CHAIRMAN';
  RAISE NOTICE 'tenant_id = %', t;
END $$;
```

1. Note the `tenant_id` from the message and create the chairman's Keycloak account with that attribute (chapter 3).
2. The chairman signs in and fills in the cooperative: Excel import, members, tariffs, meters.

> **A TEMPORARY WAY**
>
> This SQL is a workaround until the operator console; creation through it is not in the audit log. On the server a "first cooperative" command replaces it (chapter 9), later the operator console.

Isolation

- The application talks to the database as `nadel_app`, which cannot bypass RLS; migrations run as `nadel_owner`.
- Every new table has an isolation test: another cooperative's data is invisible even to a direct query.
- Deleting a cooperative is not provided; the data stays until an archiving decision (phase 7).

5 The platform operator partly

The operator is the person who runs the platform (you). The principle (ADR 0008): **the operator does not see cooperative data** — no members, no plots, no money.

What	State
Signing in as `operator@nadel.local`: the app says "No cooperative", data is unavailable	works
Operator console: list of cooperatives, creating a cooperative and its chairman, blocking	phase 7
Platform reports without personal data: number of cooperatives, plots, active users, storage used, subscription state	phase 7
Support access: a chairman grants temporary access to their cooperative; every action goes to their audit log	phase 7
Subscriptions and payment for using the platform	phase 7

Until then the operator's technical tasks are Keycloak (chapter 3), creating cooperatives (chapter 4), the server and backups (chapter 9).

6 Cooperative admin: screens and buttons

Every screen, every button — what it does and which permission it needs. A person does not see buttons they have no permission for; the server refuses such a request anyway.

Header and menu

Element	What it does	Permission
Cooperative name	Goes to the home page	—
УКР / ENG	Interface language, remembered in the browser	—
System / Light / Dark	Theme	—
Sign out	Ends the session	—
Menu	Shows only the sections the person has permissions for	—

Plots

Element	What it does	Permission
Search, "Kind", "Status", "Show archived"	Filter the register	View plots (a member sees a short list)
Add plot	Form: number, street, kind, area (sotkas or m²), cadastral number, building, notes	Edit plots
Card → Edit	Changes the plot's data	Edit plots
Card → Archive / Restore	Hides the plot from the list or brings it back	Edit plots
Card → Delete	Only for a plot that never had an owner	Edit plots
Card → Change owner	Date, new owners with shares (sum = 1), basis; "Add member" right in the form	Edit ownership
Card → Correct (next to a record)	Fixes dates, share or person in an ownership record, with a reason	Edit ownership
Card → Tariffs (month)	What the plot will be charged for the chosen month	View tariffs, or own plot
Card → Meters	The plot's meters with history and replacement chain	View meters, or own plot

Members

Element	What it does	Permission
Search, "Show archived"	By name, phone or email	View members
Add member	Name, phones, email, address, preferred channel, Telegram, Viber, notes	Edit members
Card → Edit	Changes the member's data	Edit members
Card → Archive / Restore	Blocks or restores sign-in; not for a plot owner or the last chairman	Edit members
Card → Change roles	Assigns roles to the member	Manage roles
Card → Change history	The audit log of this member	View audit log

Roles

Element	What it does	Permission
Add role	Name, description, set of permissions	Manage roles
Edit	Changes name, description, permissions (the chairman's permissions never change)	Manage roles
Restore standard name / description	Undoes the renaming of a base role	Manage roles
Delete	Only a custom role assigned to nobody	Manage roles

Cooperative settings

Element	What it does	Permission
Who sees plot colours	"Own plots only" or "All plots"	Manage settings
Meter tree	Opens the tree (below)	View meters
Tree → From / To	A range: every meter shows consumption, "calculated" or "no reading after …"	View meters

Element	What it does	Permission
Tree → Move	A new parent meter (or "outside the tree") from the date in "Effective from"	Manage meters

Import

Element	What it does	Permission
Download template	The Excel template in the interface language	Run import
Choose file → Check file	Checks without saving: new, updated, unchanged, problems by sheet, row and column	Run import
Import	Saves everything at once (only with no problems, within 30 minutes)	Run import
Import history	Who, when, which file, how many records	Run import

Tariffs

Element	What it does	Permission
Charge types → Add / Change	Name, description, "Active" (an inactive type gives no amounts)	Manage tariffs
Charge types → Up / Down	Order (later the payment allocation priority)	Manage tariffs
Charge types → Archive	Removes the type and its tariffs	Manage tariffs
New tariff	Charge type, name, first version: valid from, basis, frequency, rate, due date, note	Manage tariffs
Card → New version	A new rate from a date; a past date shows a recalculation warning	Manage tariffs
Card → Edit / Delete a version	Only while no charge uses it; the only version cannot be deleted	Manage tariffs
Card → Plots → Change	"All plots" with exclusions (with a reason) or "Selected plots"	Manage tariffs

Meters

Element	What it does	Permission
Filters, "Show removed meters"	Resource, kind, street	View meters
Register a meter	Kind, resource, plot or name, serial number, installation date, initial reading	Manage meters
Card → Edit	Serial number, name, note	Manage meters
Card → Replace meter	Date, old final reading, new serial and initial reading; "Fill in as a wrap-around"	Manage meters
Card → Remove	Removal date and final reading	Manage meters
Card → Delete	Only a meter without readings	Manage meters
Card → readings → Correct / Delete reading	While the reading is not used in a charge	Enter readings

Readings and approvals

Element	What it does	Permission
Round date	The date for all readings of the round; defaults to today, with a hint	Enter readings
Resource, Street, "No reading since"	Round filters; the resource is remembered	Enter readings
Meter row → Save	Saves the reading (the row's date can be changed); shows consumption and warnings at once	Enter readings
To approve → Approve	A member's reading becomes counted	Approve readings
To approve → Reject	With a reason the owner sees	Approve readings
View charges	yes	yes — —
Run charges	yes	— — —
Reverse charges	yes	— — —
Close periods	yes	— — —

Charges

Element	What it does	Permission
Charges → Months → month	Totals, the check list, history of the month	View charges
Month → Charge the month	Preview, then "Book"	Run charges
Month → Close the month	Confirms with the check list; only the earliest open month	Close periods
Not charged → Retry	Retries one item	Run charges
Losses / Common consumers → From, To, Preview, Book	A range run: preview, then booking of exactly that preview	Run charges
Past runs → Details, Reverse the run	The lines of a run; reversal with a reason	View charges / Reverse charges
Plot card → Account → Details	The calculation of a charge	View charges (or own plot)
Account → Reverse	A reversal with a reason (not offered in a closed month)	Reverse charges
Account → Enter an opening balance	Charge type, amount, date, reason	Run charges
Plot card → Water → Connect / Disconnect	A water connection with its dates	Manage meters
Settings → red threshold, Losses, Common consumers, Water supply	Thresholds, losses mode, splits, bulk water connections	Manage settings / Manage meters

KNOWN GAPS

- The lasting connections of a removed well can only be ended on each plot's card.
- Archiving a plot or making it common land does not end its lasting water connection: end it on the plot card first.

Audit log and account

Element	What it does	Permission
Audit log → record type, From, To, "Only this record", "Only this author"	Filter entries; "Remove filter"	View audit log

Element	What it does	Permission
My account	My details, roles, what I may do, my plots with shares, what I will be charged	—
My meters → Submit a reading	A reading for the board to approve	own plot

7 Permissions and roles

Permission	Chairman	Treasurer	Member	Electrician (demo)
View members	yes	yes	—	—
Edit members	yes	—	—	—
Manage roles	yes	—	—	—
View audit log	yes	yes	—	—
View plots	yes	yes	—	—
Edit plots	yes	—	—	—
Edit ownership	yes	—	—	—
Run import	yes	—	—	—
Manage settings	yes	—	—	—
View tariffs	yes	yes	—	—
Manage tariffs	yes	—	—	—
View meters	yes	yes	—	yes
Manage meters	yes	—	—	—
Enter readings	yes	—	—	yes
Approve readings	yes	—	—	—

- The chairman always holds every permission, including any new one.
- The chairman changes the treasurer's and other roles' permissions in "Roles".
- A member without permissions sees only their own data — enforced by the server.
- Charges: "View charges" opens the Charges screens and every plot account (the treasurer has it, read-only); "Run charges" runs months, losses and common consumers, retries "Not charged" and enters opening balances; "Reverse charges" reverses a charge or a run; "Close periods" closes a month. A member sees only their own accounts, whatever the permissions.

8 Data, security, personal data

Isolation	Database RLS on every table with cooperative data; the application cannot bypass it.
Access	The server checks permissions on every request; a member sees only their own plots.
Audit log	Who, when, what changed, old and new value. Entries are never changed or deleted.
Not stored	Tax numbers (РНОКПП), passport data, dates of birth — there are no such fields.
Server logs	No personal data: even a database error message does not include values.
Events	Completed events are deleted from the event table at once, because they may carry personal data.
Secrets	Only from environment variables; the repository holds only local development passwords. CI scans for leaked secrets (gitleaks).
Dependencies	CI checks known vulnerabilities (pnpm audit, OSV) and the code (Semgrep).

ENCRYPTING PERSONAL DATA

A separate feature is under discussion: encrypting contacts, addresses and notes in the database, so that even a copy of the database without the key reveals no personal data. On the server the disk and the backups are encrypted as well (chapter 9).

9 Server deployment planned · 6a

The decision (ADR 0010): the whole product is finished locally first, the server comes last. The plan is as follows.

What runs on the server

- One virtual server in an EU data centre (initially Hetzner Cloud, Germany or Finland), about 4 vCPU and 8 GB RAM; a smaller second one for staging.
- Docker Compose: PostgreSQL + PostGIS, Keycloak in production mode, the backend and Caddy (the web client, HTTPS with automatic certificates). Only ports 80 and 443 are open to the outside.
- Cost: tens of euros per month.

How updates arrive

1. Every merge to `main`: GitHub builds the images, pushes them to the GitHub Container Registry and updates staging.
2. A version tag `v…`: deployment to production after your approval in GitHub, with a database backup right before.
3. Rollback: deploy the previous tag with a separate action.

First installation

- The server: SSH keys only, no root, a firewall, automatic security updates.
- A domain and DNS pointing to the server; secrets in the server's environment file and in GitHub environments; the backend refuses to start with a default password.
- A "first cooperative" command creates the cooperative, its chairman and the Keycloak account and sends the set-your-password email.
- The Keycloak admin console and the database only through an SSH tunnel.

Backups and monitoring

- Every night an encrypted copy of the application and Keycloak databases, stored outside the server; 30 daily and 12 monthly copies kept. Every month a copy is restored on staging as a test.
- An external uptime check with Telegram alerts; error tracking without personal data; disk, memory and certificate alerts.

A step-by-step runbook with exact commands joins this chapter with the deployment feature.

10 Updates and database migrations

- The database structure changes only through Flyway migrations (`backend/src/main/resources/db/migration`), run when the backend starts. Existing migrations are never edited — only new ones are added.
- "Expand, then contract": a release may add tables and fields; removing or renaming waits for the next release, when the old code no longer runs. The previous version thus works on the new database and a rollback is safe.
- Demo data (`db/demo`) loads only in the development profile; the server never has it.

11 How the system is developed

From an idea to main

1. **A feature document** in `docs/features/NNN-….md`: requirements, rules, worked examples, acceptance criteria. The number is the highest in `context/spec` + 1.
2. **The pipeline**: `/feature-pipeline docs/features/NNN-….md` in Claude Code. Agents write the specification, the plan, the code and the tests slice by slice; a separate reviewer checks every slice; at the end every criterion is verified, screenshots taken, the full suite run, a PR opened and its checks brought to green.
3. **Your check**: the app from the PR branch is running for you; try the scenarios from the report.
4. **Merging** — only you: `gh pr merge <number> --squash --delete-branch`, then `git switch main && git pull`.

CI checks on every PR

Check	What it does
Backend	Build, every test with a database container, module boundaries, coverage floor
Clients	API contract, formatting, types, tests, lint, web build
Security	Leaked secrets (gitleaks), dependency vulnerabilities (pnpm audit, OSV)
SAST	Static code analysis (Semgrep)
End-to-end	Browser scenarios on a freshly started database

Rules

- Agents never merge or approve PRs and never push to `main`; only the owner commits changes to `CLAUDE.md` and `.claude/`.
- Commits never mention AI. All repository text is English, except the guides in `docs/guide` (Ukrainian and English).
- Verification levels: a new feature runs the full suite locally; a small fix on an open PR runs only the affected tests, and CI runs the full suite.
- Models: the orchestrator and the reviewer use your Claude Code session's model (Opus); the worker agents use the model set in `.claude/agents`.
- Working notes outside git: `NOTES.local.md` (questions and answers) and `DECISIONS.local.md` (decisions and deferred tasks).

12 Common problems and fixes

Problem	Cause and fix
"Port 8080 was already in use", the web opened on 5174	A previous run is still going (for example the one the pipeline left for review). `lsof -ti :8080 -ti :5173 | xargs kill`, then `make up`.
"Account not linked"	The email in Keycloak differs from the member card's, or the member is archived, or Keycloak has no `tenant_id`.
`admin` cannot sign in to the Keycloak console	Use the **master** realm at `/admin/`, not nadel; check the keyboard layout.
"E2E ..." entries and odd plots in the audit log	Traces of old end-to-end tests. Reset the local database (chapter 2); the tests now use a separate database.
"platform (linux/amd64) does not match" for postgres	The PostGIS image is Intel-only and runs emulated on a Mac — slower, tests included. An arm64 image is to be tried locally.
Slow backend tests, long CI	Backend tests are the longest part; measuring and speeding them up is planned.
Git: "File exists ... .lock"	A lock file left by an interrupted command. Make sure no git is running and delete the named `.lock`.
Realm file changes did not apply	The realm is imported only into an empty Keycloak — reset the local database.
IntelliJ keeps asking for access to other apps' data	System Settings → Privacy & Security → Full Disk Access → IntelliJ IDEA, restart.
The pipeline stopped	`/feature-pipeline resume` continues where it stopped, from its log.

13 Decision log

Date	Decision
17.10.2026	Design: the "calm registry" — side menu, tables, green accent.
18.10.2026	The chairman's answers: the chairman chooses how common consumers are split; the quarter report shows readings, cost and amount; a building is reference only; no meter, no charge; the chairman configures the meter tree.
18.10.2026	Other members' plot colours hidden by default; "All plots" only by a general meeting decision.
20.10.2026	The whole product on the local machine first, servers last.
21.10.2026	The interface always in one language. A rate has up to 4 decimals, shown without trailing zeros; amounts 2 decimals.
21.10.2026	A tariff version can be corrected or deleted until a charge uses it; then through recalculation.
21.10.2026	Readings are not tied to months: a reading is a value on a date; moves in the tree take effect from a date.
22.10.2026	A resource icon; a plot's two meters in one card; calculated consumption rounded for display.
22.10.2026	Electricity charged per reading; the chairman chooses the losses mode; negative losses are not charged; common consumers as a separate line.
22.10.2026	Water like electricity: an unconnected plot pays neither for water nor for the pump; any number of wells and quarters.
22.10.2026	Verification levels: small fixes run only the affected tests locally, the full suite in CI.
22.10.2026	Two guides in Ukrainian and English in `main`: for the board (working in the system and the calculation rules) and for the platform owner (this one).